

CS285 LECTURE 1 • ZERO-TO-HERO FIELD MANUAL

Foundations of Reinforcement Learning & The Closed Loop

From Absolute Beginner to Master: Intuition, Parameter Anatomy, Everyday Analogies, and The Core Mathematics of Decision Making

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Focus:** Closed-Loop Control, MDPs, POMDPs, & The Modern RL Landscape

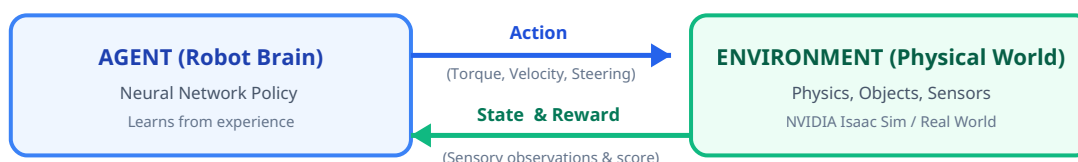
1. What is Reinforcement Learning? (The "Zero-to-Hero" Intuition)

Imagine you want to teach a toddler how to ride a bicycle. How would you do it?

- **The Supervised Learning Way:** You sit the child at a desk and show them 10,000 flashcards. Card 1: *"If you lean 2° left, contract your right quadricep by 14 Newtons."* The child memorizes all the cards, sits on the bicycle, wobbles 2.1° to the left, enters a situation not on any flashcard, panics, and crashes.
- **The Reinforcement Learning Way:** You put a helmet on the child and hold the seat. The child pushes the pedals. When they balance and coast forward, they feel a rush of excitement (**Reward**). When they wobble and scrape their knee, it hurts (**Penalty**). Nobody gave the child mathematical formulas for gyroscopic precession. Through *trial, error, and feedback*, the child's brain automatically builds an intuitive control policy.

💡 THE CORE PHILOSOPHY: LEARNING BY INTERACTION

Reinforcement Learning is not about memorizing answers. It is about learning consequences. In supervised learning, an external teacher tells you the "correct answer" for every input. In RL, there is no teacher. There is only a *scorekeeper* (the environment) that gives you a numerical reward or penalty after you act. The agent must figure out which actions caused the reward.



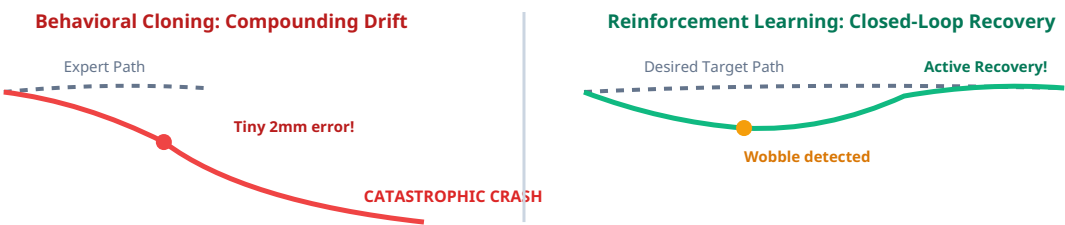
2. Why Not Just Copy Human Experts? (The Imitation Trap)

A common beginner question is: *"Why don't we just record a human expert operating the robot and train a neural network to mimic the human?"* This is called **Behavioral Cloning (Imitation Learning)**. While it sounds simple, it suffers from a fatal mathematical problem: **Covariate Shift**.

🚗 THE PASSENGER IN THE RACECAR ANALOGY

Imagine you spend 100 hours watching a Formula 1 champion drive around a track. The champion drives perfectly, staying exactly on the racing line. You memorize every steering motion.

Now you get behind the wheel. On turn 3, your hands shake slightly, and the car drifts **just 5 cm onto the dirt shoulder**. You freeze. Why? **Because in 100 hours of watching the champion, you never once saw what to do on the dirt shoulder!** The champion never made a mistake, so you never learned how to recover from one. You slide into the wall.



2.1 The Mathematics of Compounding Error: Ross & Bagnell (2011)

Ross & Bagnell formally proved why open-loop imitation learning fails over long horizons T :

Open-Loop Imitation Error $\leq \frac{1}{2}\epsilon T^2 = \mathcal{O}(\epsilon T^2)$ vs. Closed-Loop RL Error $\leq \epsilon T = \mathcal{O}(\epsilon T)$

Symbol	Formal Name	Plain English Meaning	Example Value	Impact of Parameter
ϵ	Per-Step Error Rate	The probability that the policy makes a small mistake on any given single timestep.	0.01 (1% chance per step)	Lower is better, but in the real world ϵ is never zero due to sensor noise.
T	Episode Time Horizon	The total number of control steps from start to finish.	500 steps (10 seconds at 50 Hz)	As T grows, open-loop error explodes quadratically (T^2), while RL stays linear!
$\mathcal{O}(\epsilon T^2)$	Quadratic Compounding	Once an error happens, all future steps are off-distribution, creating a snowball effect.	For $T = 100$: ≈ 50 total mistakes!	Guarantees failure in complex robotics tasks like peeling, slicing, or peg insertion.
$\mathcal{O}(\epsilon T)$	Linear Bounded Error	Because the policy learned closed-loop recovery, each error is corrected immediately.	For $T = 100$: ≈ 1 mistake total!	The fundamental reason RL succeeds where behavioral cloning fails.

 **PLAIN ENGLISH TRANSLATION OF THE EQUATION**

"If a robot does not know how to correct its own mistakes, a 1% error at step 10 will ruin the entire 500-step task (T^2). Reinforcement learning forces the robot to practice recovering from mistakes, keeping total errors strictly proportional to time (T)."

3. The Language of RL: The Markov Decision Process (MDP)

To solve problems with math, we need a standardized language. Every reinforcement learning problem on Earth is modeled as a **Markov Decision Process (MDP)**, defined by a 6-tuple: $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \rho_0 \rangle$.

Symbol	Formal Name	Plain English Meaning	Real-World Robotics Example (Tomato Slicing)
$\mathcal{S}$	State Space (s_t)	The complete snapshot of everything happening in the environment right now.	33 numbers: Dual-arm joint angles, knife position, blade contact forces, acoustic burst.
$\mathcal{A}$	Action Space (a_t)	The choices available to the robot at this microsecond.	6 numbers: Downward feed rate, sawing velocity, stiffness delta ΔK , damping delta ΔD .
$\mathcal{P}$	Transition Dynamics $P(s_{t+1} \mid s_t, a_t)$	The rules of physics. <i>"If the world is in state s and you take action a, where will you end up next?"</i>	Isaac Sim PhysX 5 solver: Computes friction, fruit deformation, and blade penetration.
$\mathcal{R}$	Reward Function $r(s_t, a_t)$	The scorecard. A single number telling the robot how well it did on this exact step.	+5.0 for cutting downward, +2.0 for sawing shear, -4.0 for crushing forces > 8 N.
γ	Discount Factor ($\gamma \in [0, 1)$)	The Patience Meter . How much does the robot care about future rewards versus immediate rewards?	$\gamma = 0.99$: The robot values completing the cut at step 300 almost as much as step 1.
ρ_0	Initial State Distribution	Where does the world start when an episode resets?	Randomized tomato placement on cutting board ± 2 cm, initial blade height 5 cm.

3.1 The Markov Property: The "Goldfish Memory" Rule

THE GOLDFISH MEMORY RULE EXPLAINED

A system has the **Markov Property** if:

$$\mathbb{P}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = \mathbb{P}(s_{t+1} \mid s_t, a_t)$$

Plain English: *"The future depends ONLY on where you are right now, NOT on how you got here."*

Imagine you are driving a car and you see a snapshot of the speedometer reading **60 mph**. Can you predict where the car will be in 1 second? **Yes!** You don't need to know if the car started in Paris, New York, or Beijing 3 hours ago. The current speed and heading contain everything needed to predict the next second.

Crucial Robotics Rule: If your robot state only included knife position $z = 4.0$ cm, you do *not* know if the blade is plunging downward at high speed or retracting upward! To satisfy the Markov property, your state **must include velocities and contact forces** ($\dot{q}, v, F_z$).

3.2 Trajectory Probability Factorization

A complete episode (trajectory) is a sequence of states and actions: $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$. Using the chain rule of probability and the Markov property:

$$\mathbb{P}(\tau \mid \theta) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

Component	Who Controls It?	Plain English Meaning
$\rho_0(s_0)$	Environment (Nature)	Where the robot starts at timestep zero.
$\pi_\theta(a_t \mid s_t)$	The Robot Brain (YOU!)	The probability that your neural network chooses action a_t given sensor state s_t .
$P(s_{t+1} \mid s_t, a_t)$	Environment (Physics)	How the physical world responds to your action. (Usually unknown and non-differentiable!).

PLAIN ENGLISH TRANSLATION OF THE EQUATION

"The total probability of an entire 500-step robot experiment is simply the probability of where it started, multiplied by every decision the robot made, multiplied by how the laws of physics responded at each step."

4. Concrete Numerical Walkthrough: 3 Steps in a Robot Slicing Episode

Let's see actual numbers! Suppose our robot is cutting a tomato with discount factor $\gamma = 0.99$.

Step (t)	Current State s_t	Action Chosen a_t	Physics Outcome & Sensor Feedback	Reward r_t
$t = 0$	$z = 5.0$ cm (Above fruit) $v_z = 0.0$ m/s, $F_z = 0.0$ N	Feed rate: +2.0 mm/s Stiffness: 1000 N/m	Knife moves downward in free air. No contact yet.	$r_0 = +0.2$ (Approach reward)
$t = 1$	$z = 4.8$ cm (Skin contact) $v_z = -2.0$ mm/s, $F_z = 3.5$ N	Feed: +1.5 mm/s Sawing: +25.0 mm/s	Lateral sawing initiates! Skin shear stress concentrates at blade edge.	$r_1 = +2.5$ (Sawing progress bonus)
$t = 2$	$z = 4.6$ cm (Tough skin resistance) $v_z = -1.0$ mm/s, $F_z = 9.8$ N	Feed: +3.0 mm/s (Too aggressive!) Stiffness: 1200 N/m	Excess force penalty! $F_z = 9.8$ N exceeds 8.0 N threshold. Tomato pulp squishes slightly.	$r_2 = -1.8$ (Pulp crush penalty)

4.1 Calculating the Total Discounted Return (G_0)

How good was this 3-step sequence for the robot? We compute the **Discounted Return** G_0 starting from step 0:

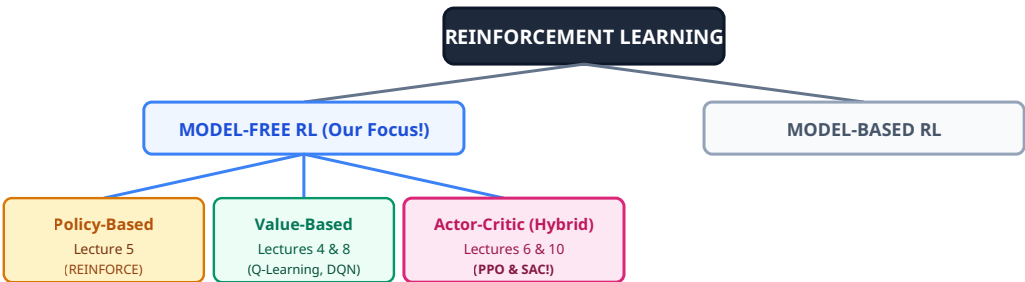
$$G_0 = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 = 0.2 + (0.99 \times 2.5) + (0.99^2 \times -1.8)$$

$$G_0 = 0.2 + 2.475 - 1.764 = +0.911$$

Why $\gamma = 0.99$? Notice that $\gamma^2 = 0.9801$. The penalty at step 2 was multiplied by 0.9801, so the robot cares almost as much about step 2 as step 0. If $\gamma = 0.1$, the penalty would be multiplied by 0.01 = -0.018, making the robot myopic and careless about future crushing!

5. The Landscape of Modern RL: Where Does Everything Fit?

When reading papers, you will hear terms like *Model-Free*, *Actor-Critic*, *Q-Learning*, *PPO*, *SAC*. Here is how they all connect:



6. The Vectorized Simulation Loop in PyTorch (Isaac Lab Style)

How does this look in real Python code? In modern robotics (NVIDIA Isaac Lab), we run **1,024 parallel environments simultaneously on one GPU**:



VECTORIZED GYM / ISAAC LAB STEPPING LOOP

```
import torch

# 1. Initialize 1,024 parallel robot environments on GPU
num_envs = 1024
states = env.reset() # Shape: (1024, 33) -> 1,024 robots observing 33 sensor channels

total_rewards = torch.zeros(num_envs, device="cuda")

for step in range(500): # 500-step episode (10 seconds)
    # 2. Policy forward pass: Robot Brain chooses 6-DoF continuous motor actions
    with torch.no_grad():
        actions = policy_network(states) # Shape: (1024, 6)

    # 3. Step physics simulation forward by 20 milliseconds (50 Hz)
    next_states, rewards, dones, infos = env.step(actions)
    # next_states: (1024, 33), rewards: (1024,), dones: (1024,) bool

    # 4. Accumulate rewards across all parallel environments
    total_rewards += rewards

    # 5. Reset environments that finished (knife hit board or tomato crushed)
    states = next_states

print(f"Average Return across 1024 robots: {total_rewards.mean().item():.2f}")
```

7. Practitioner's Failure Modes & Debugging Checklist

When training your first RL agent in Isaac Lab, 90% of failures stem from these three classic beginner traps:

Failure Mode	Why It Happens	How to Fix It Immediately
1. Observation Scale Explosion	Joint angles are in radians $[-3, +3]$, but contact forces are in Newtons $[0, 50]$ N. The neural network ignores joint angles and fixates on the large force numbers.	Standardize observations: Use a running mean and variance wrapper (e.g., RunningMeanStd) so every state dimension has mean 0 and variance 1.
2. The Discount Factor Horizon Trap	Setting $\gamma = 0.9$ means the effective planning horizon is only $H = \frac{1}{1-\gamma} = \frac{1}{0.1} = 10$ steps. The robot cannot plan a 300-step cut!	Set $\gamma = 0.99$ for slicing tasks ($H = 100$ steps), or $\gamma = 0.995$ for long manipulation trajectories.
3. Partial Observability (POMDP) Trap	Passing only blade position z without velocity $\dot{z}$ violates the Markov property. The agent cannot distinguish moving down from retracting up.	Always include first-order velocities ($\dot{q}, v_{\text{knife}}$) and historical observation buffers (e.g., stacking last 3 frames).